



Rajshahi University of Engineering and Technology
Department of Computer Science & Engineering

Course Title : Software Engineering Sessional
Course Code : CSE 3206
Experiment No : 02
Date of Submission : 18 August 2026

Submitted By:	Submitted To:
Name : Apon Datta (2203019), Emon Islam (2203020), Sanjida Tabassum(2203021) Group : 7 Section : A Series : 22	Name : Emrana Kabir Hashi Designation : Assistant Professor Department : CSE Institution : RUET

Team Information

Group 07, Section A

Name	Roll	Role	Branch	Module Responsibility
Apon Datta	2203019	Scrum Manager / Lead Developer	feature/auth-user-management	Base React architecture, AppContext.jsx state engine, AuthModal.jsx authentication, ProfileModal.jsx profile & availability, RBAC security
Emon Islam	2203020	Frontend Feature Developer 1	feature/donor-search-requests	DonorDirectory.jsx search directory, DonorCard.jsx, EmergencyRequestModal.jsx, MyRequests.jsx tracker
Sanjida Tabassum	2203021	Frontend Feature Developer 2	feature/dashboard-inventory	MetricsOverview.jsx analytics, InventoryMatrix.jsx (8 blood groups), AdminApprovalQueue.jsx, ActivityLog.jsx audit trail

Project Title & Overview

LifeFlow - A Real-Time Blood Donation Matching & Reserve Monitoring Platform

LifeFlow is a role-aware web platform that connects voluntary blood donors, patients in urgent need & blood bank administrators across Bangladesh in a single coordinated system. The platform's core purpose is to compress the time between a blood emergency and a matched, available donor while giving hospital and blood-bank authorities a live view of reserve stock across all eight standard blood groups.

Rather than relying on scattered phone calls and unmoderated social media posts, LifeFlow centralizes donor discovery, emergency broadcasting, request tracking & inventory oversight behind a single, responsive & user-friendly interface. The system is built as this lab's Minimum Viable Product (MVP): a functional, modular slice of a larger system .

Problem Statement

Bangladesh's blood donation ecosystem is fragmented and largely informal. When a patient requires an urgent transfusion, families typically resort to phone trees, hospital notice boards & public Facebook posts. This produces four recurring, serious problems:

- **Rare blood group scarcity:** Groups such as O- and AB- are statistically rare in the population, and no centralized directory exists to locate available donors quickly when they are needed most.
- **Opaque hospital reserve stock:** Blood bank inventory levels are rarely visible to the public or to other facilities, so patients cannot know in advance whether a hospital can actually fulfil a request.

- **Public phone-number privacy risk:** Donor contact numbers are frequently broadcast openly on social media, exposing donors to repeated unsolicited calls, spam, and privacy violations long after a single request is resolved.
- **Delay from social media broadcasting:** Emergency posts depend on organic reach and shares; there is no structured escalation, filtering, or tracking, so critical minutes are lost while a request circulates informally.

LifeFlow addresses each of these failure modes directly: a structured, filterable donor directory, a live inventory, role-based access that shields personal contact details from the public & a trackable emergency-request workflow with defined states.

Project Objectives

- Reduce donor-discovery response time by replacing manual phone/social-media search with an instant, filterable donor directory targeting sub-second client-side search.
- Provide a visual, at-a-glance reserve matrix covering all 8 blood groups (A+, A-, B+, B-, AB+, AB-, O+, O-) so shortages are immediately visible.
- Introduce emergency urgency classification (e.g. Critical / High / Normal) so admins and donors can triage multiple simultaneous requests.
- Enforce role-based access control (RBAC) so that only request owners or admins can modify or fulfil a request, and personal contact data is never exposed to unauthorized users.
- Demonstrate a functioning Agile Scrum execution across a 3-person team with three distinct, independently verifiable sprints.

Stakeholder Analysis

Stakeholder	Interest / Need	Primary Interactions
Donors	Control their own availability, avoid unsolicited contact, view donation history	Toggle availability, respond to targeted requests, update profile
Patients / Requesters	Find a matching donor fast, track status of their emergency request	Submit emergency requests, browse donor directory, monitor MyRequests
Blood Bank Authorities	Understand regional supply/demand, coordinate with hospitals	View inventory matrix, cross-reference pending requests
System Admins	Moderate requests, keep inventory accurate, ensure platform integrity	Approve/reject requests, adjust stock levels, review audit log

Functional Requirements

1. **User Registration:** The system shall allow a new user to register with name, contact information, blood group, and desired role (Donor / Recipient).
2. **Authentication:** The system shall authenticate returning users via a login modal (AuthModal.jsx) and maintain an active session.
3. **Role Adaptation:** The interface shall dynamically adapt visible actions and navigation based on the authenticated user's role (Donor, Recipient, Admin).
4. **Donor Listing:** The system shall display a searchable directory of registered donors via DonorDirectory.jsx, rendered as DonorCard.jsx components.
5. **Multi-Criteria Filtering:** Users shall be able to filter the donor directory by blood group, availability status, and location simultaneously.
6. **Emergency Request Submission:** Recipients shall be able to submit an emergency blood request specifying blood group, urgency level, and unit quantity via EmergencyRequestModal.jsx.
7. **Targeted Requests:** Emergency requests shall be broadcast to donors whose blood group and availability match the request criteria.
8. **Request Status Tracking:** Requesters shall be able to track the lifecycle of their submitted requests (Pending → Approved → Fulfilled / Rejected) via MyRequests.jsx.
9. **Personal vs System View Toggle:** The system shall allow a user to switch between a personal view (own requests/history) and a system-wide view (all activity, admin only).
10. **Access Control Security:** The system shall restrict request cancellation and fulfillment actions to the request creator or an Admin via a canManageRequest guard.
11. **Inventory Matrix:** The system shall display current reserve stock for all 8 blood groups via InventoryMatrix.jsx, updating reactively as stock changes.
12. **Admin Stock Controls:** Admins shall be able to increment or decrement blood-unit stock counts per blood group directly from the inventory matrix.

Non-Functional Requirements

1. **Performance:** Donor directory filtering shall complete in under 50ms for datasets up to several hundred donor records (client-side filtering, no network round trip).
2. **Usability:** The UI shall use Plus Jakarta Sans and Inter (Google Fonts) with a glassmorphism visual language for a modern, legible experience.
3. **Responsiveness:** The layout shall render correctly across viewport widths from 360px (mobile) to 1536px.
4. **Persistence:** Session, request & inventory state shall persist across page reloads via browser localStorage.
5. **Security (RBAC):** All state-mutating actions shall be gated behind role checks; unauthorized attempts shall fail silently in the UI (no destructive action exposed).
6. **Reliability:** The application shall not crash or lose in-memory state when switching between roles or views during a single session.
7. **Modularity:** UI features shall be decomposed into independent components (Directory, Requests, Inventory, Admin) to support parallel development and future testing.
8. **Visual Layout Integrity:** Badges, buttons, and Lucide icons shall use whitespace-nowrap and shrink-0 to prevent text wrapping or icon clipping at any supported width.
9. **Scalability:** The React Context data layer shall be structured so it can later be swapped for a networked backend (REST/GraphQL) without rewriting UI components.

10. **Portability:** The application shall run in any evergreen browser without platform-specific dependencies, using only Vite + React + Tailwind CSS.

User Stories & Use Cases

User Story 1: Emergency Blood Search (Recipient)

As a recipient, I want to search for available donors by blood group and location, so that I can quickly find someone who can help in an emergency.

Acceptance Criteria:

- Given a recipient is on the Donor Directory page
- When they filter by blood group "O-" and enable the "Available only" toggle
- Then only available O- donors are displayed within 50ms, sorted by relevance

User Story 2: Donor Availability Toggle (Donor)

As a donor, I want to mark myself as unavailable temporarily so that I am not contacted when I cannot donate.

Acceptance Criteria:

- Given a logged-in donor is viewing their profile (ProfileModal.jsx)
- When they toggle their status from "Available" to "Unavailable"
- Then their card no longer appears in directory searches with the "Available only" filter enabled

User Story 3: Emergency Request Broadcast (Requester)

As a requester, I want to submit an emergency request with urgency level and unit quantity so that matching donors are notified promptly.

Acceptance Criteria:

- Given a recipient opens EmergencyRequestModal.jsx
- When they submit blood group, urgency, and required units
- Then the request enters a "Pending" state and appears to matching donors and the Admin Approval Queue

User Story 4: Request Access Control & Fulfillment (Request Owner)

As a request owner, I want to be the only non-admin user who can cancel or mark my request fulfilled, so that no one else can tamper with it.

Acceptance Criteria:

- Given a request exists that was created by User A
- When User B (not the owner, not an Admin) attempts to cancel or fulfil it
- Then the canManageRequest guard blocks the action and the UI renders View Only mode for User B

User Story 5: Admin Blood Bag Allocation (Admin)

As an admin, I want to approve pending requests and allocate blood units from stock, so that fulfilled requests accurately reduce the reserve.

Acceptance Criteria:

- Given an Admin views the Admin Approval Queue with pending requests
- When they approve a request and allocate the required units
- Then the request status changes to "Approved"/"Fulfilled" and InventoryMatrix.jsx decrements the corresponding blood group count

User Story 6: Stock Matrix Monitoring (Public / Admin)

As any user, I want to see current reserve levels for all 8 blood groups at a glance, so that I understand whether a shortage exists before submitting a request.

Acceptance Criteria:

- Given any user opens the dashboard
- When InventoryMatrix.jsx renders
- Then all 8 blood groups display current unit counts with low-stock groups visually flagged

Selected Software Process Model: Agile Scrum

LifeFlow is developed using the Agile Scrum process model. The team operates in short, fixed-length sprints, each producing a demonstrable, integrated increment of the product. A lightweight backlog is maintained and reprioritized between sprints based on what is learned from the previous increment.

Sprint Backlog Breakdown

Sprint	Focus	Owner
Sprint 1	Core Store & Auth Module — AppContext.jsx state engine, AuthModal.jsx, ProfileModal.jsx, RBAC foundations	Apon Datta
Sprint 2	Donor Directory & Emergency Requests — DonorDirectory.jsx, DonorCard.jsx, EmergencyRequestModal.jsx, MyRequests.jsx	Emon Islam
Sprint 3	Dashboard & Inventory Matrix — MetricsOverview.jsx, InventoryMatrix.jsx, AdminApprovalQueue.jsx, ActivityLog.jsx	Sanjida Tabassum

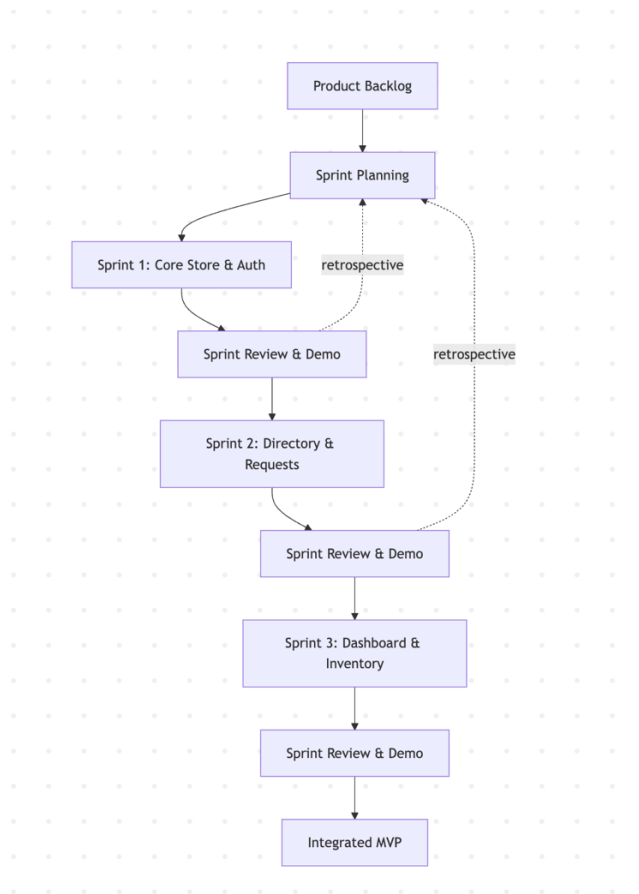


Fig 1: Sprint Process Flow

Justification of Selected Process Model

- **High-fluidity lab environment:** Requirements for a MVP evolve as feedback arrives from instructors and demos; Scrum's sprint-review builds that feedback loop in directly.
- **Rapid MVP prototyping:** The 4-week lab timeline maps naturally onto 3 short sprints, each ending in a working, demonstrable slice rather than a single big-bang delivery.
- **Parallel team collaboration:** The three module boundaries (Auth/State, Directory/Requests, Dashboard/Inventory) map cleanly onto three sprint owners who can work concurrently on separate feature branches with minimal merge contention.
- **Continuous verification:** Each sprint ends with an integration and demo step, catching interface mismatches between AppContext.jsx consumers early rather than at the end of the project.

Comparison with Alternative Process Models

Metric	Agile Scrum	Waterfall	Spiral	RAD
Requirement Stability Needed	Low	High	Medium	Low-Medium
Feedback Frequency	Every sprint	Only at the end	Every risk-analysis cycle	Frequent, prototype-driven
Team Size Fit	Ideal for small teams	Better for larger, siloed teams	Suited to large, risk-heavy projects	Small teams, tight timelines
Time-to-first-demo	Days (end of Sprint 1)	Weeks/months (end of implementation)	Long, risk analysis overhead	Fast but less structured
Documentation Overhead	Lightweight, living backlog	Heavy, upfront specification	Heavy, risk documentation each cycle	Light but weak audit trail

Why Waterfall Was Less Suitable

Waterfall requires the full requirement set to be frozen before design begins. LifeFlow's scope which is an RBAC model with three distinct roles and a multi-stage request lifecycle. It was refined through discussion and small experiments (e.g. deciding what "View Only" mode should look like) that only became clear once early UI was in front of the team. A rigid, single-pass Waterfall sequence would have forced lock-in on decisions like the shape of canManageRequest, with no cheap way to revise them once implementation began.

Why Spiral Was Less Suitable

Spiral's strength is systematic risk analysis on projects with significant technical or business uncertainty. LifeFlow is a 3-person, single-sprint-cycle MVP with modest technical risk. The overhead of formal risk-assessment cycles at every iteration would have consumed a disproportionate share of the limited lab timeline without a corresponding reduction in actual project risk.

MVP Design Overview

Technology Stack

- Frontend Framework: React 18 + Vite
- Styling: Tailwind CSS with custom glassmorphism overlays .
- Typography: Plus Jakarta Sans & Inter (Google Fonts)
- Iconography: Lucide React (lucide-react)
- State & Persistence: React Context API (src/context/AppContext.jsx) synchronized with browser localStorage for session, requests & inventory

UI Component Breakdown

Component	Responsibility
AppContext.jsx	Central reactive state engine: session, donors, requests, inventory; syncs to localStorage
AuthModal.jsx	Login / registration modal, role selection
ProfileModal.jsx	Profile editing, availability toggle
DonorDirectory.jsx	Searchable, filterable donor listing container
DonorCard.jsx	Individual donor summary card
EmergencyRequestModal.jsx	Emergency request submission form
MyRequests.jsx	Personal request tracker with status timeline
MetricsOverview.jsx	Aggregate analytics widgets (dashboard)
InventoryMatrix.jsx	8-blood-group reserve stock grid, admin-editable
AdminApprovalQueue.jsx	Admin-only queue of pending requests for approval/rejection
ActivityLog.jsx	Audit trail of RBAC-relevant actions

Role-Based Access Control (RBAC) Summary

- Donor: toggle availability (Available / Unavailable), view own donation history
- Recipient: submit emergency requests, track personal requests
- Admin: Admin Approval Queue access, blood unit allocation, inventory stock adjustment (+ / -)
- canManageRequest guard: restricts cancellation/fulfillment to the request creator or an Admin; all others see View Only mode

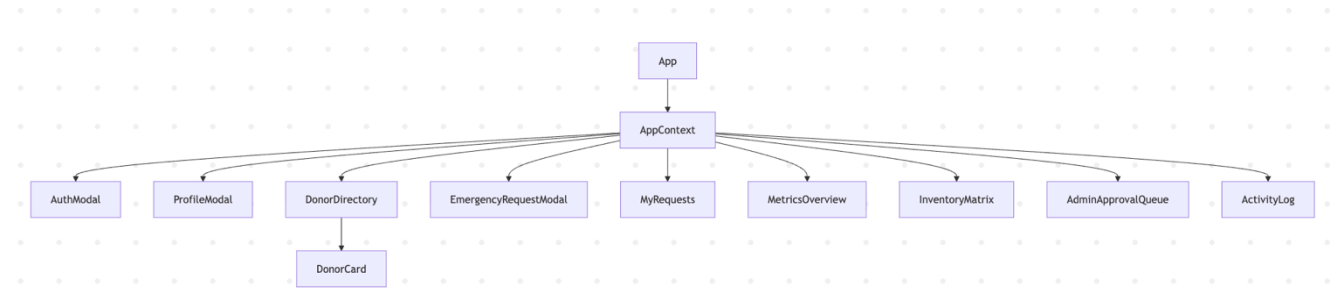
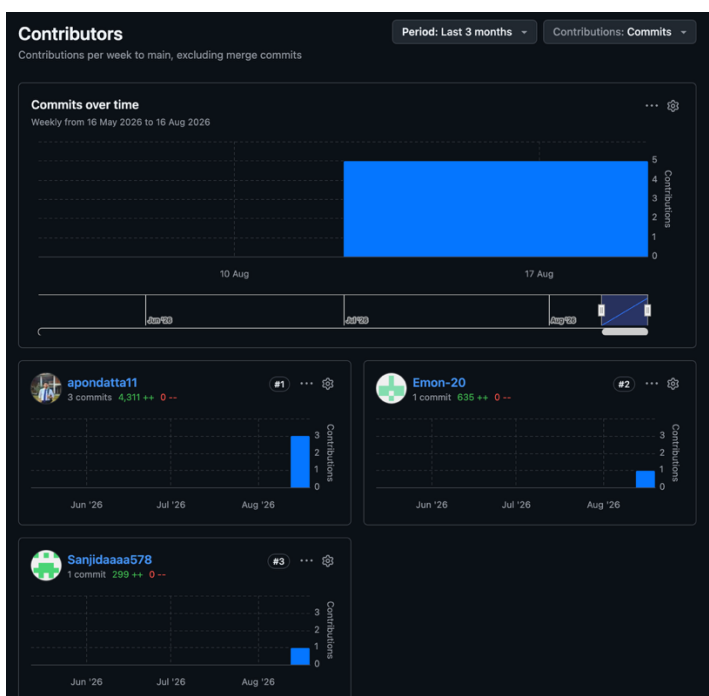
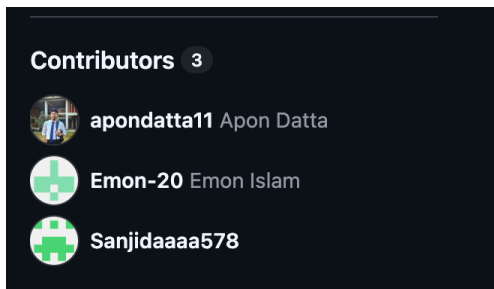


Fig 2: Component Hierarchy

GitHub Collaboration Evidence

Github Repository Link : https://github.com/apondatta11/blood_donation_management



Repository Directory Structure

```
lifeflow/  
├── README.md  
├── docs/  
│   └── Requirement Report.pdf  
├── src/  
├── assets/  
└── screenshots/
```

Branching Strategy

Branch	Owner	Purpose
main	All (protected)	Stable, reviewed, integrated codebase
feature/auth-user-management	Apon Datta	AppContext.jsx, AuthModal.jsx, ProfileModal.jsx, RBAC core
feature/donor-search-requests	Emon Islam	DonorDirectory.jsx, DonorCard.jsx, EmergencyRequestModal.jsx, MyRequests.jsx
feature/dashboard-inventory	Sanjida Tabassum	MetricsOverview.jsx, InventoryMatrix.jsx, AdminApprovalQueue.jsx, ActivityLog.jsx

Pull Request Review Protocol

- Each feature branch opens a PR against main once its module is functionally complete and self-tested.
- At least one other team member reviews and approves before merge; review comments must be resolved, not just dismissed.
- Merge conflicts are resolved locally by the PR author, re-tested & then re-pushed for a final review pass.
- No direct pushes to main & all changes land through a reviewed PR.

Challenges Encountered & Engineering Mitigations

Challenge 1: Layout Overflow & Badge Icon Clipping

Badges and Lucide icons in the donor cards and status pills would wrap or clip at narrower viewport widths, breaking the glassmorphic card layout.

Mitigation:

- Constrained the page container to max-w-[1536px]
- Applied whitespace-nowrap to all badge and button text
- Applied shrink-0 to every Lucide icon so flex containers never compress it below its natural size

Challenge 2: Unauthorized Request Modification

Early builds allowed any authenticated user to cancel or mark any request fulfilled which is a serious data-integrity and trust issue for an emergency-coordination tool.

Mitigation:

- Introduced a canManageRequest guard function checked before rendering any mutating control
- Non-owner, non-admin users are shown a read-only "View Only" state instead of action buttons

Challenge 3: Admin Approval Queue Visibility

The Admin Approval Queue initially rendered an empty card shell for non-admin users and for admins with zero pending requests, cluttering the dashboard.

Mitigation:

```
if (!isAdmin || pendingRequests.length === 0) return null;
```

This early-return guard ensures the component renders nothing at all unless there is both an authorized admin viewer and at least one pending request to act on.

Conclusion

In lab 2, we delivered a functioning MVP of LifeFlow: a role-aware donor directory, an emergency request workflow with RBAC-guarded state transitions & a live 8-blood-group inventory matrix, all built on a React + Vite + Tailwind stack with a centralized Context state engine. The three-sprint Agile Scrum breakdown allowed us to work on independent feature branches in parallel, integrating through reviewed pull requests into main.